

OnOff业务用户访谈洞察报告

USER RESEARCH · OPERATING MODEL DIAGNOSTIC

On/Off 业务用户访谈洞察报告

核心流程、时间结构、交接物、用户旅程、信息使用与结构性痛点

核心判断 On/Off 团队的主要约束不是编码速度，而是遗留系统知识重建、跨层级职责下沉和增量测试闭环不完整。需求分析与开发都存在 2 天至 1 个月的高波动，但当前缺少可解释的工作量证据链。

研究对象	On/Off Domain 的 PO / 架构 / 开发复合角色
业务范围	FO 需求承接、影响分析、Spike、拆卡、开发、自测、Domain Test 与系统测试衔接
访谈时间	2026 年 8 月 6 日；有效讨论约 60 分钟
证据基础	单一受访者深访；包含经验估算与现场展示信息
报告日期	2026 年 8 月 7 日

分析口径：明确区分“受访者陈述”“访谈现场推演”“咨询推断”和“待验证项”。

1. 管理层摘要

一句话结论 On/Off 的端到端交付由“需求理解—遗留系统盘点—方案共创—拆卡—开发—自测—版本验证”构成；真正的重成本集中在开发前的知识重建与方案收敛，而质量风险集中在增量需求缺少独立、及时的集成验收。

六项核心发现

01 流程不是线性的 开发在正式拆卡前即通过 Spike 介入方案；PO 同时承担架构和部分开发职责，角色随问题切换。

02 典型开发为 2-3 周 编码/适配可短至 1-2 天，也可长至约 1 个月；典型值是 1-1.5 个双周 Sprint，约 2-3 周。

- 03 知识重建是主要成本** 项目早期，对十几个仓库和既有逻辑的盘点可占约 50%；人员不与 Domain 固定绑定，使熟悉度难以持续累积。
- 04 职责与依赖设计下沉** FO 层的需求常只有一句话，系统间集成关系和技术依赖需要 PO↔PO 再次拉齐，未完成的工作继续下沉至架构/开发。
- 05 增量测试存在断层** 开发自测承担增量功能的主要质量保证；Domain/System Test 更偏版本稳定性与回归，新增用例可能滞后一至两个 Sprint。

优先级判断

优先级	议题	管理判断
P0	增量质量闭环	明确开发自测、Domain Test、集成测试和 System Test 的对象与时点；建立当 Sprint 的增量验收。
P0	<u>工作量可解释性</u>	<u>把 Spike、依赖澄清、仓库盘点、开发、联调和测试分别计量，解决“2 天到 1 个月”无法解释的问题。</u>
P1	遗留知识与人员绑定	形成仓库/模块知识地图、熟悉度矩阵和交接机制；减少人员切换导致的重复冷启动。
P1	需求与依赖治理	将 FO 层应完成的系统依赖、接口契约和验收口径显式化，避免工作无边界下沉。

2. 业务范围、角色与协作边界

On/Off 是车载软件中的一个系统级 Domain，与诊断、OTA 等其他系统处于平级关系，但存在依赖和交互。在 Middleware 组织中，同一团队覆盖约 12 个大 Domain；On/Off 自身又涉及 QNX、VCPU、Android 等多个技术域和十几个代码仓库。

访谈所呈现的角色模型

角色	名义职责	现实中的扩展职责	主要协作对象
车级 PM / 子 PM	按车型上市节点倒排季度/PI 目标与里程碑。	在资源和依赖冲突时进行升级协调。	FO、各 Domain PM
FO	承接系统需求，协调平级系统依赖，向下传递需求。	成熟度不一；部分需求只有一句话，集成方案未完全定义。	其他 FO、PO、上级 PM

PO	理解 FO 需求、分解任务、安排 Sprint。	兼架构、需求分析、问题初判和部分开发；跨 PO 拉齐技术依赖。	FO、其他 PO、架构、Dev
架构	评估技术可实现性、定义软件架构与接口方案。	与 PO 高度重合；开发通过 Spike 深度参与。	PO、各技术域 Dev
Dev	增量开发、适配、联调与开发自测。	提前参与方案、盘点遗留逻辑；缺陷进入时协助 PO 快速判断。	PO/架构、其他 Dev、测试
Domain Test	执行 Domain 级版本稳定性验证。	单个迭代约 2-3 天；并非完整的当期增量集成验收。	Dev、集成/System Test

组织特征 角色并非严格分工，而是围绕问题动态切换。灵活性提高了兜底能力，也削弱了职责边界、成本归属和交付可解释性。

3. 核心工作流程（Current State）

流程从 FO 系统需求开始，在 PO/架构层完成影响分析和方案收敛，并通过 Spike 让开发提前介入。开发完成后，质量验证分为开发自测、Domain Test 与后续集成/System Test，但三者的验证对象并不相同。



图 1 | On/Off 业务端到端工作流（基于访谈重建）

流程拆解与主要交接物

环节	主责/参与	关键动作	主要交接物
1. 系统需求输入	FO / PM	承接系统级需求，提出跨系统依赖；按季度/PI里程碑倒排。	系统需求、依赖票、里程碑
2. 影响分析	PO + 架构	理解需求；拉 QNX、VCPU、Android 等 Domain；盘点十几个仓库与遗留实现。	疑问清单、影响范围、初步架构
3. Spike / 方案	PO/架构 + Dev	验证技术路线、实时性、资源、接口、分辨率/车型差异等；开发提前验证可落地性。	Spike 卡、技术方案、验证结论
4. 拆卡与排期	PO	在 Sprint 初期完成细化，安排后续 2-3 个 Sprint；更远工作做粗排并暴露依赖风险。	Sprint 卡、工作量、依赖/风险
5. 增量开发	Dev	代码梳理、修改/新增、适配多个仓库与车型；必要时跨团队联调。	代码、配置、联调结果
6. 开发自测	Dev	验证当期增量功能，检查日志和接口行为；是增量需求质量的主要即时保障。	自测结果、日志、问题记录
7. Domain Test	专职测试	对 Domain 版本稳定性与既有用例进行验证，访谈估计每迭代约 2-3 天。	测试结果、缺陷
8. 集成/System Test	集成/系统测试	以版本稳定性和回归为主；新用例可能在后续 1-2 个 Sprint 才进入测试集。	回归结果、缺陷、验收反馈
9. 缺陷闭环	测试 →PO→Dev	追踪跨版本/跨供应商问题；定位后修复、重新集成并回归。	Bug 单、修复卡、版本记录

关键决策点

决策点	若为 “简单”	若为 “复杂”
既有架构能否直接承载？	复用现有逻辑，可能只改数行代码，1-2 天交付。	进入多仓库盘点和架构调整，前期分析可达 1 个月。
是否涉及跨 Domain / 跨系统？	由 On/Off 内部完成拆分与开发。	需要 FO↔FO、PO↔PO 双层依赖拉齐，并建立依赖票。
开发是否熟悉相关仓库？	知识已建立，Spike 和定位成本下降。	重新冷启动，既有逻辑盘点可能占早期工作约 50%。
当期增量是否进入测试集？	在后续回归中被持续验证。	短期主要靠开发自测，集成风险可能延迟暴露。

4. 时间占比与周期结构

量化口径 访谈提供了阶段周期和若干比例锚点，但没有完整工时台账。以下将“明确陈述”与“现场推演”分开，不能直接用于绩效或预算。

受访者明确给出的周期

阶段	典型/范围	解释
需求到架构/实现方案确定	通常 1-3 周；范围 1-4 周	复杂度来自遗留系统、多仓库、跨 Domain 依赖和技术可行性。 历史上无 AI 时有过约 2 个月。
增量开发与适配	典型 2-3 周；范围 2 天-1 个月	复用既有架构可短至 1-2 天；全新增量、联调或多模块适配可达 1 个月。
Domain Test	每迭代约 2-3 天	验证 Domain 版本稳定性，不应等同于当期增量需求的完整集成测试。
计划节奏	双周 Sprint；季度/PI 预排	一个月约两个 Sprint；方案阶段通常安排后续 2-3 个 Sprint。

PO角色 时间占比：可支持的最稳健表达

工作类别	占比判断	证据强度与说明
遗留逻辑盘点 / 可行性分析	项目早期可接近 50%	受访者明确确认；团队熟悉仓库后会下降，不能作为全周期固定比例。
PO 需求拉齐 + 架构方案	访谈现场估算约 40%	受访者在现场白板中给出“上面两趴约 40%”，但原表未保留，建议仅作方向性估算。
票据/系统录入与管理	存在显著成本，未形成可靠比例	受访者认为开票与管理“挺耗时”，现场数字讨论不完整，不建议强行量化。
开发/测试/协调	其余时间动态分配	PO、架构、Dev 角色重叠，当前无法仅靠访谈拆出互斥且合计 100% 的比例。

咨询判断 报告不构造虚假的 100% 饼图。更可信的做法是连续跟踪 2-3 个 PI/Sprint，将“主动工作、等待、会议、知识检索、返工”分别计时。

5. 交接物与质量断点

交接方向	现有交接物	主要质量问题	建议最小标准
车级 PM → 子 PM/FO	车型里程碑、季度/PI 目标	里程碑明确，但容量与依赖风险要到 PI 规划时再暴露。	目标、截止时间、容量假设、关键依赖
FO → PO	系统需求、依赖票；部分场景仅一句话	系统集成关系和技术边界可能未定义，需求含义需 PO 再推导。	系统交互、上下游、接口契约、验收口径
PO/架构 → Dev	Spike 卡、Sprint 卡、口头/会议结论	方案在共创中形成；若未沉淀，工作量与设计理由不可追溯。	影响仓库、方案决策、风险、AC、估点
Dev → Test	代码、开发自测结果、版本	当期增量主要靠开发自测；测试集纳入滞后。	功能用例、预期结果、日志点、回归范围
Domain Test → 集成/System Test	稳定性结果、缺陷、新增回归用例	不同测试层的边界被混称；责任和时点不一致。	分层测试矩阵、用例 Owner、纳入 Sprint
缺陷 → PO/Dev	Bug 单、问题会议、跨版本追踪	历史责任跨供应商，问题可能半年后才暴露，定位和归责困难。	首次引入版本、复现条件、证据、Owner

四类关键交接物

- a. 需求包：系统目标、交互范围、依赖系统、接口契约和可验证 AC。
- b. 方案包：遗留现状、影响仓库、架构选择、Spike 结论和未决风险。
- c. 交付包：代码/配置、联调版本、开发自测证据和已知限制。
- d. 质量包：分层测试用例、纳入回归的 Sprint、失败证据和关闭标准。

6. 用户旅程：PO / 架构复合角色视角

阶段	用户目标	关键行为	体验/风险	改进机会
接收需求	弄清 FO 到底要什么	阅读需求、补问上下文、确认系统依赖。	信息薄，担心遗漏	需求就绪度检查
盘点遗留系统	确认现有架构能否承载	查十几个仓库、代码与历史实现，拉各 Domain 讨论。	高认知负担；冷启动	知识图谱与影响分析
形成方案	在安全和可实现性前提下拍板	与 Dev 做 Spike，验证接口、时序、性能和资源。	谨慎；怕后期卡住 1-2 个月	自动证据与方案模板

拆卡排期	把方案转成可执行工作	建 Spec/Dev/Test 卡，安排 2-3 个 Sprint，暴露依赖风险。	管理成本高；估点波动	自动拆卡与估算基线
开发与联调	确保多仓库、多域增量可工作	编码/适配、跨 PO 协调、处理临时问题。	角色频繁切换	依赖看板和上下文助手
自测与交付	证明当期增量正确	开发自行功能测试、记录日志并交付版本。	质量责任集中于开发	测试草案与证据回填
等待回归	确认集成后版本稳定	等待 Domain/集成/System Test，在后续 Sprint 接收反馈。	反馈延迟；风险后移	当 Sprint 增量门禁
缺陷回流	定位跨版本、跨团队问题	召集会议、追溯供应商与历史版本、修复回归。	高打断；归责争议	版本追溯和日志分诊

旅程低谷 “盘点遗留系统”是认知与时间成本最高的阶段；“等待回归—缺陷回流”是质量风险最集中的阶段。

7. 关键信息及其使用方式

信息对象	包含什么	如何被使用	主要风险
车型/PI 里程碑	上市倒排、季度目标、Sprint 节点	决定需求优先级、人员投入和风险上报时点。	容量变化靠加人/拉长周期消化
FO 系统需求	系统级功能意图与少量依赖	作为 PO 影响分析和跨系统讨论的起点。	可能只有一句话，缺集成设计
依赖票	其他系统对 On/Off 或反向依赖	建立正式承诺；未提票时可拒绝执行。	发现过晚会影响排期
遗留代码与仓库	十几个仓库、既有时序/资源/接口逻辑	判断可复用性、确定改动点和方案风险。	人员不绑定，知识反复重建
架构/Spike 结论	技术路线、时序、性能、车型差异、可行性	支撑拆卡、估点和后续开发决策。	口头化会导致理由不可追溯
Sprint 卡与估点	Spec、开发、联调、测试任务和工时	资源计划、进度跟踪、成本解释。	阶段混合，无法解释波动
测试用例与结果	开发自测、Domain 回归、系统回归	证明功能与版本稳定性，发现缺陷。	测试层级边界与纳入时点不清

版本/缺陷历史	供应商、交付版本、首次失败时间、修复记录	跨半年追溯问题，判断归属和修复范围。	链路断裂导致甩锅和复盘困难
人员技能/熟悉度	Domain、仓库、模块知识和角色能力	分配人员、判断分析周期和风险。	未系统化盘点，不支持估算

信息使用的核心模式

- 里程碑信息用于“何时必须完成”；遗留代码与架构信息用于“能否完成、怎么完成”。
- 依赖票用于建立跨系统承诺；PO↔PO 讨论用于补足 FO 层未定义的技术实现。
- Sprint 卡既是执行载体，也是成本解释载体，但当前没有把 Spike、知识重建和返工显式拆开。
- 测试用例和版本历史用于质量背书；由于反馈滞后，其价值更多体现在事后发现，而非当期预防。

8. 核心痛点与根因诊断

级别	痛点	访谈证据	业务影响	根因
P0	增量需求测试断层	开发自测当期增量；Domain/System Test 偏稳定性；新用例可能后移 1-2 Sprint。	集成问题延迟暴露，半年后仍可能追溯历史交付。	测试对象、Owner 与时点未分层
P1	职责逐层下沉	FO 需求可能只一句话；系统集成方案由 PO/Dev 补齐；无人处理时逐级拉领导。	高技能人员做大量澄清与协调，方案责任模糊。	角色定义存在但产出标准缺失
P1	跨系统依赖重复对齐	FO↔FO 和 PO↔PO 都要沟通；依赖未在上游充分表达。	会议与等待增加，排期风险后置。	依赖契约不完整、缺统一视图
P2	管理性录入耗时	开票、系统录入和文档管理被明确认为成本高。	挤压方案与指导时间，且数据未必形成洞察。	工具碎片化与重复录入

根因链

上游需求和依赖未完全结构化 → PO/Dev 承担额外系统分析 → 依赖遗留代码与个人经验形成方案 → 任务估点波动且难解释 → 增量测试仍以开发自测为主 → 问题在后续版本回归中延迟暴露 → 跨团队追责和返工进一步占用关键角色。

9. 对数字员工 / Agent 平台的启示

产品原则 优先建设“需求—依赖—仓库—方案—测试—版本”的可追溯链，并把 Agent 用在影响分析、知识重建和质量编排上；单纯代码生成不是最大价值点。

机会	能力描述	预期价值	前提/风险
1. 需求与依赖审查 Agent	从 FO 需求识别缺失的系统交互、依赖票、接口契约、验收口径，并生成澄清清单。	减少职责下沉与重复会议	需定义 FO/PO 最小交接标准
2. Spike 共创 Agent	根据需求与历史实现生成方案草案、风险清单、验证步骤和决策记录。	提高方案一致性和可追溯性	必须保留工程师审批与安全门禁
3. 分层测试编排 Agent	生成开发自测与增量集成用例；将新用例在当 Sprint 纳入 Domain/系统回归。	补齐增量质量闭环	需测试环境和自动化接口
4. 管理证据 Agent	自动汇总 Spike、等待、沟通、开发、测试和返工时间，形成容量与风险报告。	让负荷和资源诉求可证	避免把数据用于简单化绩效考核

不建议优先做

- 只做通用代码补全：不能解决知识重建、依赖治理和测试滞后。
- 把一个 Test Agent 同时覆盖开发测试、Domain Test、集成测试和 System Test：四者对象与责任不同。
- 在没有数据治理时直接做精确人效预测：当前角色重叠、卡片混合，容易制造虚假精度。

10. 建议行动路线图

阶段	目标	关键动作	衡量指标
0-4 周	建立事实基线	抽取 2-3 个 Sprint；把分析、知识检索、Spike、开发、联调、等待、测试、返工分开记录。	各阶段中位时长、等待占比、估算偏差
0-4 周	明确测试分层	定义 SWE.4/5/6 或对应层级的对象、Owner、输入/输出；新增功能当 Sprint 建测试责任。	增量用例及时纳入率、缺陷发现阶段
4-8 周	规范最小交接包	FO→PO 和 PO→Dev 使用统一模板；依赖票、接口契约、AC、影响仓库与风险成为必填项。	需求就绪率、澄清轮次、返工率
4-10 周	建设知识与影响分析 MVP	索引 12 个 Domain、仓库、模块、Owner、历史方案和测试；先覆盖 On/Off 高频场景。	检索命中率、方案形成时长、新人上手时长

8-12 周	上线 Agent 辅助	需求审查、相似实现检索、拆卡草案、测试草案和证据汇总，均保留人工审批。	节省工时、采纳率、误报率
12 周后	闭环质量与容量	把新增用例自动纳入回归；用实际数据校准区间估算和人员熟悉度矩阵。	逃逸缺陷率、周期波动、容量预测准确度

建议的北极星指标

从“FO 需求进入 PO”到“当期增量获得可复核、分层测试证据”的中位时长 该指标同时覆盖需求就绪度、知识重建、方案收敛、开发、联调和增量质量闭环，比单独统计代码量或票数更接近真实交付价值。

11. 待验证问题与研究局限

下一轮访谈应验证

- FO：系统需求、跨系统依赖和验收口径在正式流程中应交付到什么深度？一句话需求的真实比例是多少？
- Dev（志宏、马婷）：Spike、编码、联调、自测、等待分别占多少时间？哪些仓库最难理解？
- Domain / 集成 / System Test：各层实际测试对象、用例纳入时点、人员规模和缺陷发现分布。
- 项目管理：卡片中的工时能否区分 Spec、知识盘点、协调和返工？过去 3 个 PI 的估算偏差是多少？
- 组织负责人：人员不与 Domain 绑定的原因、收益与代价，是否存在稳定 Owner 或备份机制？
- 工具/数据：需求、依赖票、仓库、版本、测试和缺陷系统能否通过 API 建立追溯链？

局限

本报告基于一次、单一受访者访谈。时间区间属于经验描述，部分比例来自现场共同推演，且现场展示的资源表未随转写保留。因此结论适合用于流程诊断、机会排序和下一轮研究设计，不应直接作为个人绩效、人力预算或供应商扣款的唯一依据。

附录：关键证据索引

时间点	访谈证据主题	本报告中的使用
02:30-05:26	FO→PO/架构→多 Domain；方案讨论 1 周至 1 个月，通常 1-3 周	核心流程、方案周期

06:43-12:42	开发 2 天至 1 个月；开机动画三段、分辨率等案例；开发参与 Spike	开发波动、方案共创
15:14-21:44	季度/PI 倒排；Spec 与后续 2-3 Sprint；早期盘点可到 50%；典型开发 2-3 周	时间结构、计划节奏
22:27-23:44	两个开发加 PO/架构；角色随时切换，PO 也处理开发与缺陷初判	角色模型、关键人风险
29:12-34:28	人力资源盘点；Dev 参与需求分析；PO/架构/Dev 投入现场推演	时间占比、可解释性
34:56-40:21	Domain Test 2-3 天；集成/System Test 偏稳定性；新回归项可后移 1-2 Sprint	测试断层
40:47-41:56	SWE.4/5/6 测试层级不能由单一 Test Agent 全吃	Agent 边界、测试治理
42:36-51:39	系统平级但存在依赖；FO↔FO、PO↔PO；依赖票；FO 需求可能只有一句话	依赖治理、职责下沉
52:05-56:54	PI 信息传递稀疏；PO 动作与约 40% 现场估算；开票/管理耗时	时间占比与管理成本
58:01-59:19	职责与流程不清、负荷无法量化、外部仅靠信任	核心痛点与证据链

资料来源：用户提供的《onoff访谈.txt》。未使用外部资料。